OIC Stock Alert - High-Level Design

Version: 1.1.0 | Date: 2026-05-26 | Status: Complete
Source Design: docs/02-design/features/oic-stock-alert.design.md
Repository: https://github.com/ai-ml-kiosk/test-oic-stock-api

---

1. Purpose

This High-Level Design describes the OIC Stock Alert service architecture from an integration and operations viewpoint. The service is a stateless HTTP API that Oracle Integration Cloud invokes to evaluate stock alert rules and route the result through stable OIC Switch branches.

The HLD focuses on system context, major components, API boundaries, data flow, security posture, and operating model. Detailed JSON schemas, field-level mapping, validation, and test cases are captured in the companion LLD:

docs/02-design/features/oic-stock-alert.lld.md

2. System Context

The OIC Stock Alert API sits between an OIC integration flow and a stock quote provider. OIC owns orchestration, scheduling, retry policies, notifications, and downstream business handling. The API owns request validation, quote retrieval abstraction, rule evaluation, response mapping, and OIC branch decision fields.

```
Actor or System | Role
Oracle Integration Cloud | Calls the API, maps JSON response fields, evaluates Switch
branches, sends notifications, and handles retries or faults.
OIC Stock Alert API | Validates payloads, fetches quote data, evaluates alert rules, and
returns branch-friendly JSON.
Quote Provider | Supplies quote data. MVP uses mock provider mode by default and adds
Alpha Vantage GLOBAL_QUOTE as the first explicit live provider path.
Notification Target | Receives alert messages from OIC when the API returns
ALERT_TRIGGERED.
Operations / Monitoring | Reviews request IDs, OIC tracking IDs, decision codes,
provider failures, and timeout signals.
```

3. Architecture Overview

The MVP is implemented as a dependency-free Python standard-library HTTP service. It keeps deployment and contract testing simple while preserving a clean split between routing, validation, provider access, evaluation, response mapping, and errors.

```
OIC Integration
  |
  | POST /v1/alerts/evaluate
  v
HTTP Route Handler
  |
  +--> Request Validator
  |
  +--> Quote Provider Abstraction
        |
```

```
        +--> Mock Quote Provider
        |
        +--> Alpha Vantage GLOBAL_QUOTE Provider
  |
  +--> Alert Evaluator
  |
  +--> Response Mapper
  |
  v
OIC-Friendly JSON Response
  |
  v
OIC Switch on response.oic.switchBranch
```

4. Major Components

```
Component | Responsibility | Implementation Module
HTTP Server | Hosts health and alert evaluation routes. | oic_stock_alert/server.py
Application Service | Coordinates validation, provider lookup, evaluation, and response
mapping. | oic_stock_alert/app.py
Request Validator | Normalizes symbols, request IDs, currency, options, and alert rules.
| oic_stock_alert/validation.py
Quote Provider | Selects mock or Alpha Vantage live provider based on runtime
configuration. | oic_stock_alert/quote_provider.py
Alpha Vantage Provider | Calls GLOBAL_QUOTE, normalizes quote fields, and maps upstream
provider conditions. | oic_stock_alert/alpha_vantage_provider.py
.env Config Loader | Parses local untracked provider settings without exposing secrets
to Git or logs. | oic_stock_alert/env_loader.py
Alert Evaluator | Applies operators to quote metrics and creates rule outcomes. |
oic_stock_alert/evaluator.py
Response Mapper | Builds success and error responses with decision and oic routing
fields. | oic_stock_alert/mapper.py
Error Model | Represents validation, provider, timeout, and system error categories. |
oic_stock_alert/errors.py
Runtime Config | Reads port, provider mode, provider name, timeout, and override
settings. | oic_stock_alert/config.py
```

5. API Overview

5.1 Health Endpoint

GET /health

Purpose:

- Confirms the service is running.
- Exposes service name, version, provider mode, and timestamp.
- Supports simple OIC and platform health checks.

5.2 Alert Evaluation Endpoint

POST /v1/alerts/evaluate

Purpose:

- Accepts a symbol, optional OIC source metadata, one to ten alert rules, and runtime options.
- Evaluates rules against provider quote data.
- Returns 200 OK for successful evaluations whether or not an alert fired.

- Returns structured error responses for validation, unsupported rule, provider, timeout, and system failures.

Primary branch field for OIC:

```
response.oic.switchBranch
```

Supported branch values:

- ALERT_TRIGGERED
- NO_ALERT
- INPUT_ERROR
- QUOTE_UNAVAILABLE
- PROVIDER_TIMEOUT
- SYSTEM_ERROR

6. Data Flow

1. OIC invokes POST /v1/alerts/evaluate with JSON payload and optional X-Request-Id header.
2. API uses the body requestId, then X-Request-Id, then generated UUID fallback.
3. Validator normalizes the symbol and currency, validates rules, and defaults optional fields.
4. Quote Provider returns normalized quote metrics from mock fixtures or Alpha Vantage GLOBAL_QUOTE.
5. Alert Evaluator compares each rule threshold with the selected quote metric.
6. Response Mapper creates aggregate decision fields and oic routing fields.
7. OIC Switch evaluates response.oic.switchBranch.
8. OIC sends a notification, records no-op, handles input issues, retries timeout paths, or routes technical faults.

7. OIC Integration Pattern

OIC should configure the REST invoke response mapping to preserve these fields:

```
Field | OIC Usage
requestId | Cross-system correlation and retry preservation.
symbol | Notification and log context.
quote.lastPrice | Notification content and audit detail.
rules[] | Triggered rule detail for alert body generation.
decision.code | Human-readable and operational decision category.
decision.severity | Notification priority and incident routing hint.
oic.switchBranch | Primary OIC Switch expression.
oic.notificationRecommended | Notification guard for alert branch.
oic.retryRecommended | Retry guard for timeout or system fault handling.
oic.trackingId | OIC instance correlation.
```

The preferred OIC Switch expression is:

```
response.oic.switchBranch
```

If nested mapping is inconvenient, map it to:

```
stockAlertSwitchBranch = response.oic.switchBranch
```

8. Security Design

The MVP uses deterministic JSON contracts and avoids secret exposure. It does not introduce authentication inside the local mock implementation; production deployment should place the API behind the chosen platform ingress, API gateway, or OIC connection security policy.

Security controls:

- Reject malformed JSON and invalid request shape.
- Enforce symbol, rule count, operator, metric, severity, and threshold validation.
- Keep provider credentials in environment variables or untracked local .env files only.
- Commit only .env.example placeholders; never commit .env or .env* secret files.
- Do not return provider credentials or runtime secret configuration in responses.
- Log request IDs, symbols, provider mode, elapsed time, and decision codes without secret-bearing values or full provider URLs.
- Preserve OIC correlation IDs for audit and incident investigation.

9. Operations Design

Operational behavior is intentionally simple:

```
Area | Design
Runtime | Python 3.11+ standard-library HTTP server.
Default port | 8080, configurable with PORT.
Provider mode | mock by default, configurable with QUOTE_PROVIDER_MODE.
Live provider | alpha_vantage through Alpha Vantage GLOBAL_QUOTE when explicitly
configured.
Timeout | Provider timeout target controlled by QUOTE_PROVIDER_TIMEOUT_MS, defaulting to
1500 ms.
Diagnostics | Included only when options.includeDiagnostics is true.
Correlation | requestId and oic.trackingId included in success and error responses.
Retry hint | oic.retryRecommended is true for provider timeout and system error paths.
```

10. Deployment View

The first repository version is optimized for local and integration-contract validation.

Deployment-ready assumptions:

- OIC can call the API over HTTPS once hosted behind the selected gateway or runtime platform.
- Provider credentials are injected as environment variables in hosted live mode or loaded from untracked .env in local development.
- Alpha Vantage GLOBAL_QUOTE requires function=GLOBAL_QUOTE, a ticker symbol, and apikey.
- Alpha Vantage quote freshness depends on API entitlement; realtime or 15-minute delayed data is not guaranteed by default.
- OIC retry policy should preserve the original requestId.
- Health checks use GET /health.
- API logs should be routed to the platform logging backend.

11. Quality and Traceability

The previous PDCA check phase matched 95% of the implemented mock-provider design. The current design extension adds live Alpha Vantage provider and .env configuration deliverables that are pending implementation.

Traceability summary:

```
Plan Requirement | HLD Coverage
Health endpoint | API Overview, Operations Design
Rule input | API Overview, Data Flow
Quote retrieval / mock mode | Architecture Overview, Major Components
Alpha Vantage live provider | Architecture Overview, Major Components, Operations Design
.env credential parsing | Security Design, Deployment View
Alert evaluation | Major Components, Data Flow
OIC-friendly response | OIC Integration Pattern
Error handling | API Overview, Operations Design
Diagnostics | Operations Design
```

12. HLD Decisions

```
Decision | Rationale
Keep service stateless | OIC remains orchestration owner and retries can safely re-call
evaluation.
Use branch field response.oic.switchBranch | Reduces OIC Switch mapping ambiguity.
Default to mock provider | Enables deterministic local tests and contract validation
without external dependencies.
Add Alpha Vantage as first live provider | Provides a concrete outbound REST integration
while preserving the existing provider abstraction.
Use untracked .env only for local secrets | Keeps API keys out of Git while allowing
local live-provider testing.
Keep response mapper explicit | Prevents accidental breaking changes to OIC routing
contracts.
Keep dependencies minimal | Makes the first API version easy to clone, test, and run.
```

13. HLD Open Questions

- Should production authentication be enforced by API gateway, OIC connection policy, or service code?
- Should quote-unavailable errors ever trigger automatic retry for provider-specific transient failures?
- Which notification channels should OIC invoke for ALERT_TRIGGERED?
- Should premium Alpha Vantage entitlement=delayed or entitlement=realtime become configurable later?